



Sella la verdad de tus fotos

Informe de Estado del Proyecto

MVP para Shipaton 2026 (RevenueCat)

Fecha del informe	6 de septiembre de 2026
Repositorio local	C:\Users\Administrator\Documents\VERITY
Rama	master (9 commits, sin remoto de GitHub configurado aún)
Deadline de submission	30 de septiembre de 2026
Categorías objetivo	Peace Prize · #BuildInPublic Award · HAMM Award

Este documento resume el estado real del MVP de Verity al cierre de la sesión de desarrollo del 6 de septiembre: qué se construyó, qué se probó en dispositivo físico, qué decisiones de arquitectura se tomaron (y por qué), qué falta, y cómo se compara contra el timeline original del hackathon.

Resumen ejecutivo

El flujo esencial de Verity —capturar una foto, calcular su huella digital SHA-256 en el dispositivo, anclarla en Polygon Amoy (testnet) y mostrar un certificado con nivel de confianza— está **implementado, probado en un dispositivo Android real y funcionando de punta a punta**. Durante las pruebas en vivo aparecieron 8 problemas reales (de dependencias, de red, y de UX), todos diagnosticados y corregidos en la misma sesión, con verificación después de cada arreglo (compilación de tipos, chequeo de dependencias y build completo del bundle).

El proyecto sigue dentro del scope acordado: sin login, sin wallet visible, sin detección de IA, sin multi-chain y sin token VRT. Dos decisiones de producto quedaron documentadas explícitamente por su impacto en la garantía de confianza del sistema: no auto-sellar desde la cámara nativa del sistema (debilitaría el nivel ALTO), y el respaldo exportable nunca incluye la foto (solo hashes).

Semáforo por área

Área	Estado
Setup técnico (Expo, hash, blockchain)	[HECHO] Completo y probado en dispositivo
Flujo esencial (cámara/galería -> sello -> certificado)	[HECHO] Completo y probado en dispositivo
Nivel de confianza (ALTO/MEDIO/BAJO)	[HECHO] Completo, corregido y verificado con casos reales
Duplicados y respaldo del historial	[HECHO] Completo
RevenueCat (freemium)	[PENDIENTE] Stub funcional, sin API keys reales
UX de fricción mínima (cámara directa, onboarding)	[PENDIENTE] Diseñado, no implementado aún
Backend / verificación sin número de sello	[PENDIENTE] Pendiente, requiere infraestructura nueva
Página pública de verificación	[PENDIENTE] Pendiente
Materiales Devpost / Google Play	[PENDIENTE] Pendiente

Avance contra el timeline original

Ventana	Hito planificado	Estado real
7–10 sep	Setup y core tech (Expo, hash, conexión a Polygon Amoy)	[HECHO] Completo
11–14 sep	Funcionalidad esencial (captura, sellado, certificado)	[HECHO] Completo (adelantado, cerrado el 6 sep)
15–18 sep	Integración RevenueCat real + UX minimalista	[PENDIENTE] Pendiente — siguiente etapa
19–22 sep	Backend simple y testeo con 5 personas	[PENDIENTE] Pendiente
23–24 sep	Cuenta Google Play (\$25) y subir app	[PENDIENTE] Pendiente
25–27 sep	Materiales para Devpost	[PENDIENTE] Pendiente
28–30 sep	Submit a Devpost	[PENDIENTE] Pendiente

El proyecto va adelantado respecto al plan original: la funcionalidad esencial (planificada para el 11–14 de septiembre) ya quedó completa y probada en dispositivo real el 6 de septiembre, dejando margen extra antes de la siguiente etapa.

Qué está hecho (verificado, no solo escrito)

Flujo esencial

- Captura por cámara o galería dentro de la app (`CaptureScreen.tsx`).
- Huella digital SHA-256 calculada 100% en el dispositivo, sin subir el archivo (`hashService.ts`, migrado a la API nueva de `expo-file-system@57`).
- Registro (anclaje) del hash en Polygon Amoy vía una wallet invisible generada y guardada en el almacenamiento seguro del sistema (`blockchainService.ts`).
- Certificado final con nivel de confianza, historial local, detalle completo dentro de la app (sin salir al navegador salvo que el usuario lo pida) y botón para sellar otra foto.

Nivel de confianza (ALTO / MEDIO / BAJO)

- Corregido un bug real: todo archivo de galería calificaba siempre como MEDIO, incluso sin metadatos EXIF reales, por un valor de respaldo (*fallback*) que enmascaraba la ausencia de datos.
- Verificado en dispositivo con 3 casos reales: foto de cámara de la app -> ALTO; foto de cámara externa con EXIF real -> MEDIO; imagen generada por IA sin metadatos -> BAJO.

Duplicados y continuidad de uso

- Antes de anclar en blockchain, se revisa el historial local; si el archivo ya fue sellado, se muestra el certificado existente en vez de gastar gas de nuevo.
- Botón "Sellar otra foto" — antes, la pantalla se quedaba trabada mostrando el primer certificado sin forma de continuar.

Respaldo y recuperación

- Exportar historial local a un archivo .j son compatible (Drive, correo, etc.) e importarlo de vuelta en otro dispositivo.
- Documentado explícitamente: el respaldo NUNCA incluye la foto, solo hashes y metadatos — por diseño de privacidad, con la limitación real de que no prueba autoría por sí solo.

Cambios implementados en esta sesión (6 de septiembre)

Registro cronológico real (con hash de commit) de todo lo corregido y construido, incluyendo los problemas encontrados probando en un dispositivo Android físico:

Hora	Commit	Cambio	Motivo / problema real encontrado
16:13	246e425	Alinear dependencias a Expo SDK 57	El proyecto mezclaba expo@57 con paquetes de la era SDK 51; Metro no arrancaba.
16:18	d1c2d40	Declarar babel-preset-expo como dependencia directa	npm no la hoisteaba a la raíz; Metro fallaba con "Cannot find module".
16:22	5f5fe51	Polyfill de randomBytes para ethers en React Native	La generación de la wallet fallaba: RN no trae crypto.getRandomValues nativo.
16:39	7666a0d	Cambiar RPC de Amoy a publicnode.com	El RPC oficial de Polygon no resolvía por DNS en el dispositivo de prueba real.
16:50	3b3d53c	Guardar fotos + botón "Sellar otra" + detalle in-app	Intento inicial con expo-media-library (galería del sistema).
16:57	14b38da	Revertir expo-media-library, usar almacenamiento propio	media-library@57 requiere development build; no funciona en Expo Go.
17:03	ed95c82	Nivel de confianza real por EXIF + botón de galería en Verificar	Todo salía "MEDIO"; el botón de galería en Verificar no daba ninguna señal de error.
17:47	9d49517	Detección de duplicados + respaldo exportable/importable	Responde a "¿qué pasa si sello dos veces?" y "¿qué pasa si pierdo el teléfono?".

Cada corrección se verificó con `npx tsc --noEmit, npx expo-doctor (21/21 checks)` y, en los cambios de mayor riesgo, un build completo del bundle con `npx expo export` antes de pedir una nueva prueba en el teléfono — para no hacer perder tiempo con intentos fallidos evitables.

Decisiones de diseño (y por qué)

¿Por qué no sellar automáticamente desde la cámara nativa del sistema?

Requeriría un servicio en segundo plano vigilando la galería (development build nativo, fuera de Expo Go) y, más importante, debilitaría la garantía del nivel ALTO: ya no habría certeza de que el archivo no fue editado entre el instante de captura real y el momento en que Verity lo detecta. Alternativa de bajo costo adoptada en el roadmap: abrir la app directo en la cámara + widget/acceso directo en la pantalla de inicio.

¿Qué pasa si se agotan los tokens (gas) para sellar?

En testnet (la demo del hackathon) el gas es gratuito vía faucet y no es un problema real. Para producción, la arquitectura ya está pensada para meta-transacciones: un relayer pagaría el gas (financiado por las suscripciones PRO de RevenueCat), manteniendo la wallet invisible al usuario — el mismo principio de diseño que ya existe, extendido también al gas.

¿El hash prueba que una foto es "la misma" tras un filtro o edición?

No, y es intencional: SHA-256 tiene efecto avalancha — cualquier cambio, por mínimo que sea, produce un hash completamente distinto. Verity prueba integridad exacta de un archivo ("este bit por bit existía en este momento"), no similitud visual. Esto es coherente con cómo funcionan los estándares reales de procedencia de contenido (C2PA / Content Credentials), que tampoco intentan resolver ese caso con un solo hash.

¿Por qué el respaldo exportable no incluye la foto?

Coherente con que Verity nunca sube el archivo original a ningún lado. Consecuencia documentada: el respaldo por sí solo no prueba autoría, solo restaura el índice local de qué se selló y cuándo — la prueba de autoría real depende de conservar el archivo original en algún lado.

Qué falta (priorizado)

Prioridad alta — etapa 15–18 sep

- Cuenta de RevenueCat real + API key de Android (el stub ya está listo para recibirla).
- Paywall visual + trial configurado para jueces.
- Abrir la app directo en la cámara al iniciar (reduce la fricción de uso reportada).
- Onboarding de 3 pantallas explicando por qué el sello se hace dentro de la app.

Prioridad media — etapa 19–22 sep

- Backend simple (backend/anchor . ts) para poder verificar un archivo sin necesitar ya el número de sello a mano.
- Testeo con 5 personas reales fuera del equipo.
- Página pública de verificación (verification/[id] . html).

Prioridad baja / fuera de este MVP

- Meta-transacciones (gas pagado por la app) — respuesta de visión para jueces, no build.
- Hash perceptual secundario para detectar "muy similar aunque no idéntico" — roadmap v2.
- Icono final de producción, screenshots de Google Play, materiales de Devpost.
- Cuenta de Google Play Console (\$25) y build de producción.

Estructura del proyecto

```
VERITY/  
|-- App.tsx                      # Entry point raiz (re-exporta app/App.tsx)  
|-- app/  
|   |-- App.tsx                 # Inicializa RevenueCat + navegacion  
|   |-- screens/  
|       |-- HomeScreen.tsx      # Contenedor de las 3 pestanas  
|       |-- CaptureScreen.tsx   # Sellar: camara/galeria -> hash -> ancla -> certificado  
|       |-- CertificatesScreen.tsx# Mis sellos: historial + exportar/importar respaldo  
|       |-- VerificationScreen.tsx# Verificar: por galeria o por "Mis sellos"  
|   |-- components/  
|       |-- CameraButton.tsx  
|       |-- TrustLevelBadge.tsx  
|       |-- CertificateCard.tsx  
|       |-- CertificateDetailModal.tsx # Detalle in-app (nuevo)  
|   |-- services/  
|       |-- hashService.ts      # SHA-256 local (API nueva de expo-file-system@57)  
|       |-- blockchainService.ts # Wallet invisible + anclaje en Polygon Amoy  
|       |-- revenuecatService.ts # Stub freemium (10/mes -> PRO $4.99)  
|   |-- utils/  
|       |-- cryptoUtils.ts      # Historial local, duplicados, respaldo (AsyncStorage)  
|-- backend/                    # Pendiente (verificacion por archivo sin numero de sello)  
|-- verification/               # Pendiente (pagina publica de verificacion)  
|-- assets/icons/app-icon.png   # Icono 1024x1024, generado localmente  
|-- documentation/
```

```
| |-- technical/verity-protocol.ts # Tipos y contrato del protocolo (fuente unica)
| |-- shipaton-submission/      # Pendiente (materiales Devpost)
|-- reference/                  # Documento maestro v1.1 y web informativa (NO se implementan)
```

Pruebas realizadas

Verificación estática (cada cambio, antes de cada prueba en dispositivo)

Comando	Qué valida	Resultado
<code>npx tsc --noEmit</code>	Compilación de tipos de TypeScript	OK — sin errores en cada iteración
<code>npx expo-doctor</code>	Salud de dependencias vs. Expo SDK 57	OK — 21/21 checks
<code>npx expo export --platform android</code>	Build real del bundle con Metro + Babel	OK — 3 builds completos (1006-1013 módulos), sin errores

Pruebas en dispositivo Android real (Expo Go)

Caso	Qué prueba	Resultado
Arranque de la app	Carga sin errores, splash con ícono correcto	CONFIRMADO
Sellar con cámara	Foto -> hash -> transacción confirmada en Polygon Amoy	CONFIRMADO (tx real en Amoy)
Certificado ALTO	Cámara de la app + GPS + hora	CONFIRMADO
Certificado MEDIO	Foto de galería con EXIF real (foto navideña)	CONFIRMADO tras el fix
Certificado BAJO	Imagen de IA sin metadatos EXIF	CONFIRMADO tras el fix
Sellar otra foto	Reset de pantalla tras un sello	CONFIRMADO
Detalle in-app	Ver certificado sin salir al navegador	CONFIRMADO
Verificar por Mis sellos	Comparar archivo local contra su propio sello	CONFIRMADO
Verificar por galería	Botón antes silencioso ahora avisa / abre picker	Corregido, pendiente reconfirmar en dispositivo
Detección de duplicados	Re-sellar el mismo archivo no genera uno nuevo	Implementado, pendiente probar en dispositivo
Exportar / importar respaldo	Generar .json y restaurarlo	Implementado, pendiente probar en dispositivo

Convención: "CONFIRMADO" = probado en el teléfono del equipo durante esta sesión. "Implementado, pendiente..." = verificado estáticamente (tipos + build), pendiente de la próxima ronda de pruebas en dispositivo.

Wallet y transacción de referencia (testnet)

Dirección de la wallet (dispositivo de prueba)	0xb46b4657B87DFefc369f5D7DedD35A02D7455ffE
Fondeo inicial	0.1 POL vía faucet oficial de Polygon (Amoy)
Transacción de sello confirmada	0xab14c1dcd0f7edb6b5b6734ee3273990fa7ff58aeec3fa44fe329964b55509d1
Bloque	46908808 – Estado: Éxito

Preguntas que un jurado probablemente hará (respuestas preparadas)

"¿Qué pasa si se acaban los tokens de gas?"

En testnet es gratuito y recargable en minutos. En producción, un relayer/paymaster pagaría el gas (financiado por RevenueCat PRO), manteniendo la wallet invisible — ya es la misma filosofía de diseño aplicada un paso más allá.

"¿Por qué tengo que abrir la app para tomar la foto? Es fricción."

Es la fricción que le da valor al sello — como notarizar un documento. Se irá reduciendo (acceso directo a cámara, widget) sin eliminarse, porque eliminarla (auto-sellado en segundo plano) rompería la garantía de "esto se selló en el instante de la captura".

"Si edito la foto con un filtro, ¿el hash sigue coincidiendo?"

No, y es correcto que no coincida: Verity prueba integridad exacta del archivo original, no similitud visual. La versión editada nunca fue sellada — sería una afirmación falsa que sí coincidiera.

"¿Qué pasa si pierdo el teléfono?"

El sello en la blockchain no se pierde nunca. Lo que se pierde es el índice local de conveniencia (miniaturas, saber qué sellaste) — mitigado con el respaldo exportable/importable, con la limitación honesta de que ese respaldo no incluye las fotos en sí.

Próximos pasos inmediatos

- Confirmar en dispositivo: verificación por galería, detección de duplicados y respaldo exportable/importable (los tres implementados hoy, sin re-confirmar aún en el teléfono).
- Crear cuenta de RevenueCat y completar la API key en .env.
- Implementar arranque directo en cámara + onboarding de 3 pantallas.
- Decidir con el equipo si el backend de verificación (etapa 19–22 sep) se construye con Vercel o Supabase, según lo definido en el stack técnico original.